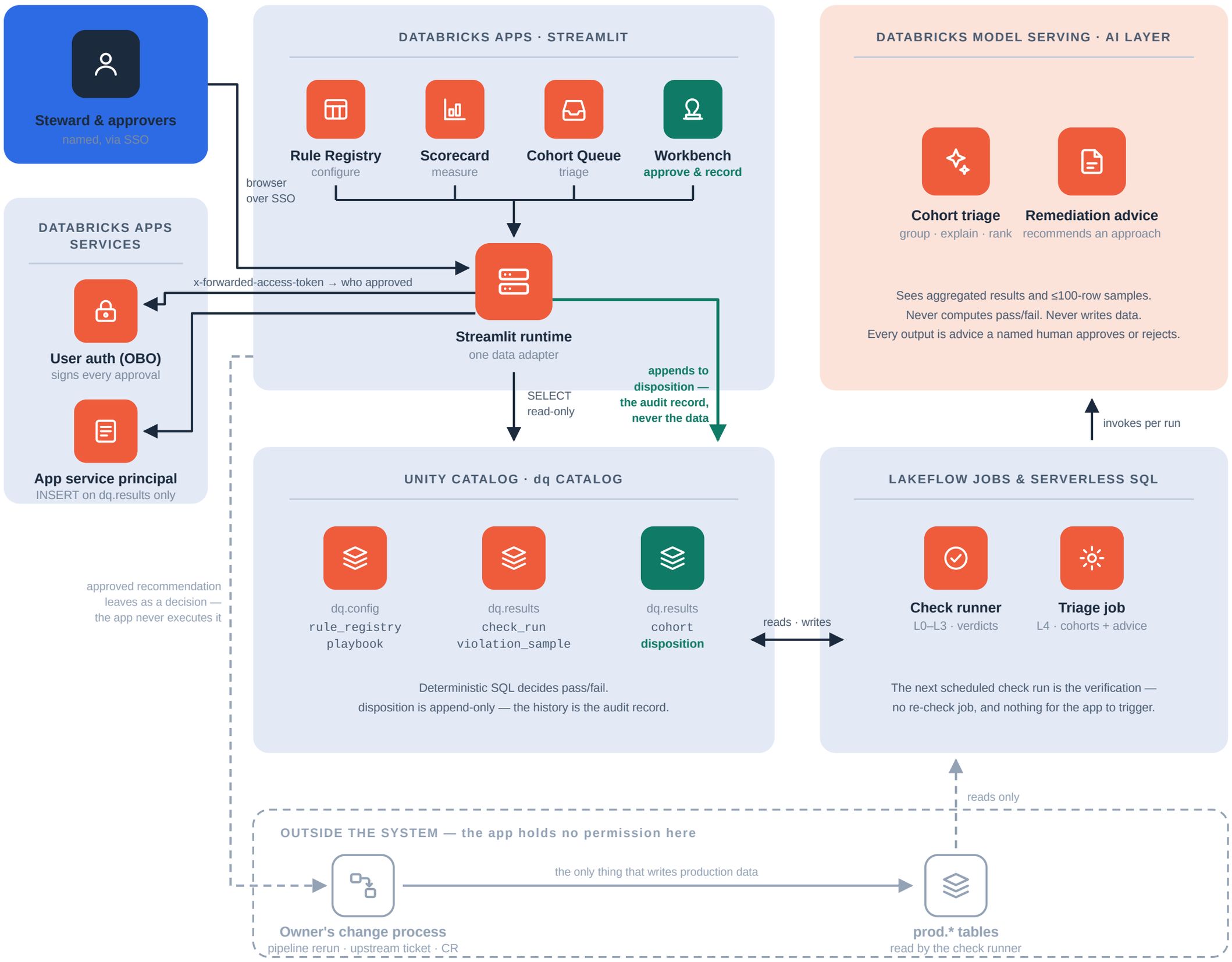
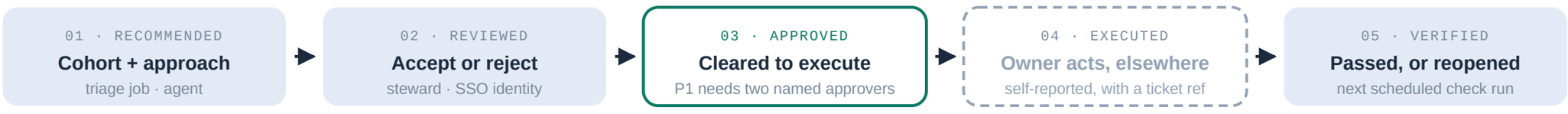
DQ Triage Agent

Detection groups into cohorts, the agent explains and recommends, named people approve, and the data owner executes — **outside this system, in their own change process.** The app's only write is the append-only record of what was found, advised, approved, done and verified.



The approval chain

Five events, each a new immutable row in `dq.results.disposition`. Nothing is ever updated in place, so the sequence itself is the evidence.



Steps 01, 02, 03 and 05 are produced inside the system. Step 04 is a claim the owner makes — the register records it, it does not witness it.

A rejected or deferred cohort ends the chain at 02 with a reason. A failed verification at 05 appends a reopen and the chain continues.

Nothing here is ever deleted or overwritten. Corrections are new rows, not edits.

Two identities, deliberately

On-behalf-of-user auth forwards the signed-in user's token, so the approver is established by the platform, not typed in. **The app's service principal** performs the write, granted `INSERT` on `dq.results` and nothing else. Identity you can trust; permission you can bound.

Unity Catalog is the enforcement

The app holds no grant on any `prod.*` table, so "it cannot change business data" is checkable by your governance team rather than a promise about the code. Under OBO the app is also confined to its declared OAuth scopes — a steward who can write to production in a notebook still cannot do it through this app.

What this control is, and isn't

It records that named people approved, and that someone reported executing. It cannot prevent execution without approval, or prove what ran matched what was approved. That makes it a **detective** control, not a preventive one. If it ever covers SOX-relevant data, prevention has to live in change management and prod grants — not here.